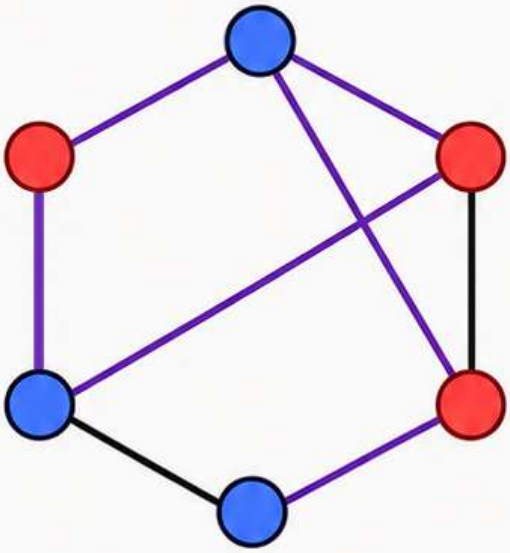
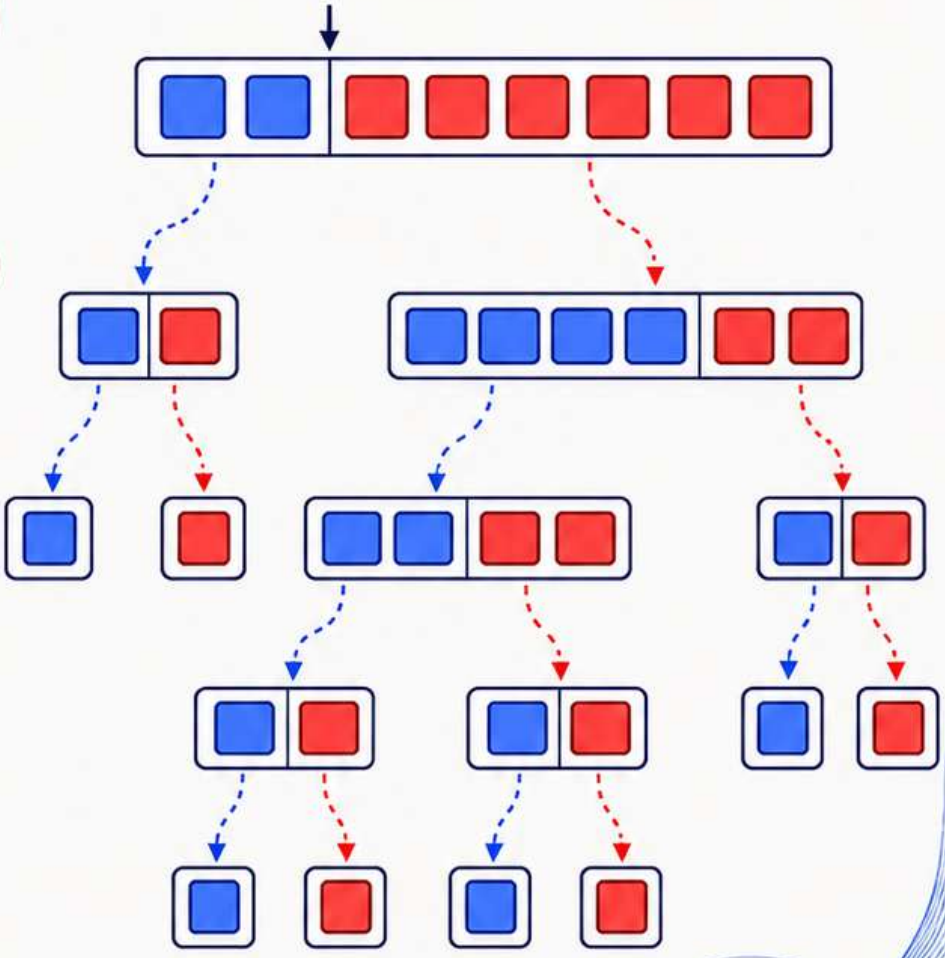
Randomised Algorithms

Sahil M Deo





Contents

Chapter 6

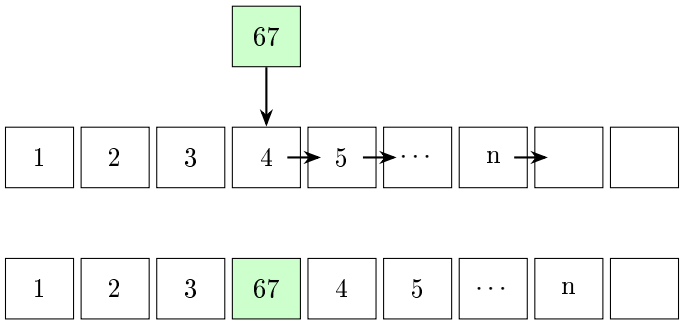
Data Structures

6.1 Skip Lists

Arrays

Suppose we store the following array in contiguous memory:

Suppose we wish to insert the element 67 between 3 and 4. Since arrays occupy contiguous memory, every element beginning from 4 must be shifted one position to the right in order to create space for the new element.

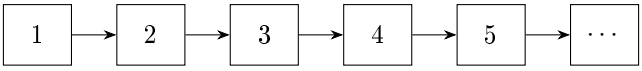


Insertion into an array requires shifting $\Theta(n)$ elements in the worst as well as average case.

How are Linked Lists any better?

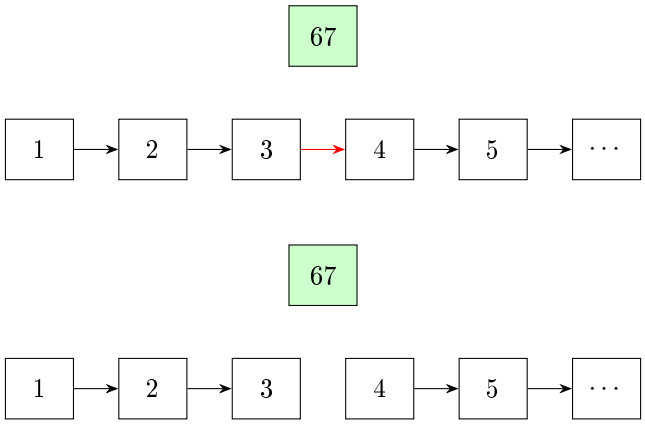
Unlike arrays, linked lists do not store elements contiguously in memory. Instead, each element stores `{Value,Next}`, where `Next` is a pointer to the next element.

So a linked list containing the elements $1, 2, 3, 4, 5, \ldots, n$ may be represented as

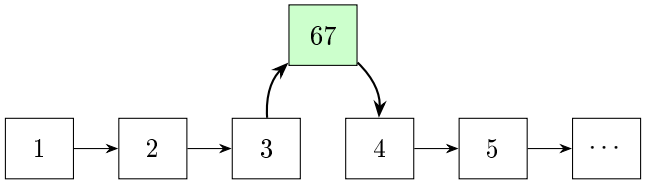


To insert an element 67 between say 3 and 4, we simply create a new node containing 67 and link it between 3 and 4.

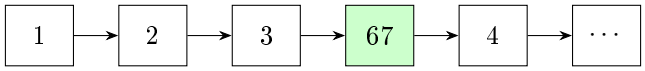
Step 1: Remove the old connection



Step 2: Create new connections



This linked list is exactly what we desired and can be equivalently represented as –



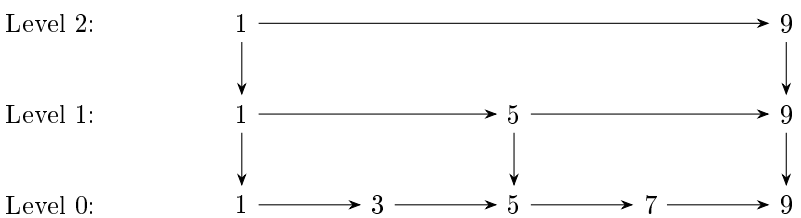
We used a constant number of operations to insert an element into the list. Thus insertion is $\Theta(1)$ time complexity.

Why Skip Lists?

Although insertion is $\Theta(1)$ time complexity, searching for an element in linked lists is $\Theta(n)$ time complexity because we have to traverse the whole list. Even if the linked list is "sorted", we can not binary search since that requires random access in constant time, which a linked list can not provide.

So the idea of skip lists is to bypass that limitation by adding “express lanes” to a linked list. Instead of traversing every element one-by-one, we maintain additional higher-level lists that skip (hence the name) over many elements at once.

Consider the following skip list –



Higher levels contain fewer elements and allow us to jump across large portions of the list quickly.

6.1.1 The Structure

The presence of an element in a level is determined by a random choice. An element occurs in level i with probability 2^{-i}. This means every element must occur in level 0.

Consequently, Pr(node reaches level k) $= \frac{1}{2^k}$. Thus, the expected number of nodes in level k is $\frac{n}{2^k}$.

Level	Expected Number of Nodes
0	n
1	$n/2$
2	$n/4$
3	$n/8$
$\vdots$	$\vdots$

6.1.2 Expected Height

Let H be the height of a skip list with n independent nodes, where each node is promoted to the next level with probability 1/2. Then it is easy to express the probability that height H is greater than or equal to h.

$$\Pr(H \geq h) = 1 - (1 - 2^{-h})^n$$

We use the tail-sum formula to find the expected value of H.

$$\mathbb{E}[H] = \sum_{h=1}^{\infty} \Pr(H \geq h) = \sum_{h=1}^{\infty} \left(1 - (1 - 2^{-h})^n\right)$$

Method 1: Splitting Method

Step 1: Split the expectation

Let $h_o = \lfloor \log_2 n \rfloor$ be the splitting point for the sum.

$$\mathbb{E}[H] = \sum_{h \leq h_0} \Pr(H \geq h) + \sum_{h > h_0} \Pr(H \geq h) = S_1 + S_2$$

Step 2: Bounding S_1

Consider the function $f(h) = \Pr(H \geq h) = 1 - (1 - 2^{-h})^n$. Since $f(h)$ is a decreasing function, we can bound it by $f(h_0)$ and $f(0)$.

$$f(h_o) \leq f(h) \leq f(0) \implies \sum_{h=1}^{h_o} f(h_o) \leq \sum_{h=1}^{h_o} f(h) \leq \sum_{h=1}^{h_o} f(0)$$

Since $f(0) = 1$, and $f(h_o) \geq 1 - (1 - \frac{1}{n})^n \geq 1 - \frac{1}{e}$ we get bounds for S_1.

$$h_o(1 - \frac{1}{e}) \leq S_1 \leq h_o$$

Step 3: Bounding S_2

Using Bernoulli's inequality[1], $(1-x)^n \geq 1-nx$ so we can bound $f(h)$.

$$1-(1-2^{-h})^n \leq n \cdot 2^{-h} \implies S_2 \leq \sum_{h>h_0} n \cdot 2^{-h} \leq 2$$

Step 4: Combining bounds

$$(1-e^{-1})h_0 + 2 \leq \mathbb{E}[H] \leq h_0 + 2$$

Since $h_0 \approx \log_2 n$, we get $\mathbb{E}[H] = \Theta(\log n)$

Method 2: Geometric Decline Theorem

Recall that using the markov inequality, we proved that the height of the recursion tree for `QuickSelect` is at most $\Theta(\log n)$

Using the same technique, we can show that if a sequence of nonnegative random variables satisfies $\frac{\mathbb{E}[X_{i+1}]}{\mathbb{E}[X_i]} \leq p$ $(0 < p < 1)$ and T denotes the first index for which $X_T \leq 1$, then

$$\mathbb{E}[T] \leq \Theta(\log(X_0))$$

Applying the Theorem

Let X_i denote the number of nodes present at level i of the skip list. Since every node is promoted independently with probability 1/2,

$$\mathbb{E}[X_i] = n\left(\frac{1}{2}\right)^i \implies \frac{\mathbb{E}[X_{i+1}]}{\mathbb{E}[X_i]} = \frac{1}{2}$$

The height of the skip list is precisely the highest level containing at least one node. Equivalently, if $H = \min\{i : X_i \leq 1\}$ then the height is H. Applying the Geometric-Decline theorem with $p = \frac{1}{2}$ gives us the desired upper bound.

$$\mathbb{E}[H] \leq \Theta(\log n)$$

This second method provokes a natural question – what was the point of the first method? In the second method we did not prove a lower bound! Although even in further analyses we won't need the lower bound, it is good to have established that it is exactly $\Theta(\log n)$. Also, we revised the splitting technique for finding the complexity of an expression, which is a useful tool in general.

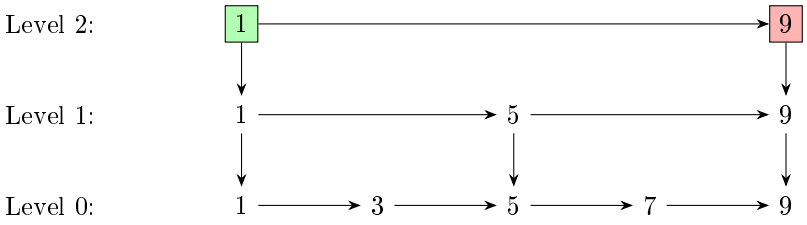
6.1.3 Search

Suppose we wish to search for the element 7. We begin at the top left level and move right while the next element is ≤ 7. Once that element is strictly greater than 7, we move down to the next level. Repeat until we reach the bottom level.

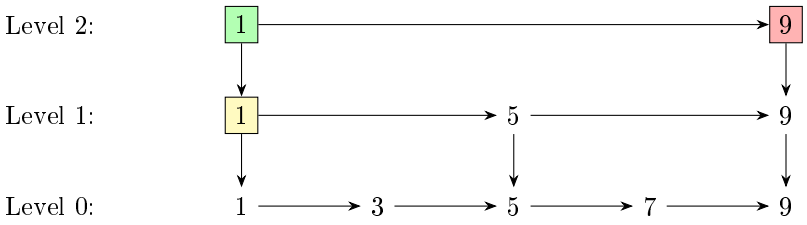
[1] Refer Appendix B.4

```
contains(x)
{
    current = top-left node
    while current exists
    {

        while current.right exists and is <= x
            current = current.right

        if current.value == x
            return FOUND

        current = current.down
    }

    return NOT FOUND
}
```

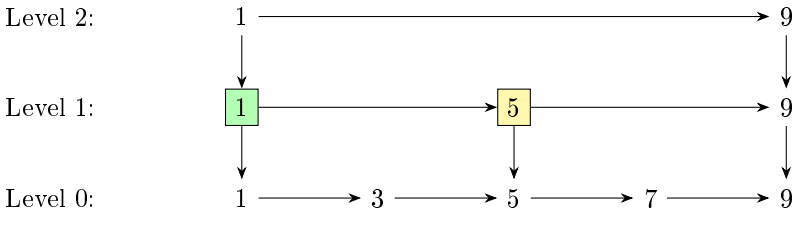
Step 1: Start at top, check right



Since on the right we have an element strictly greater than 7, we intend to move down.

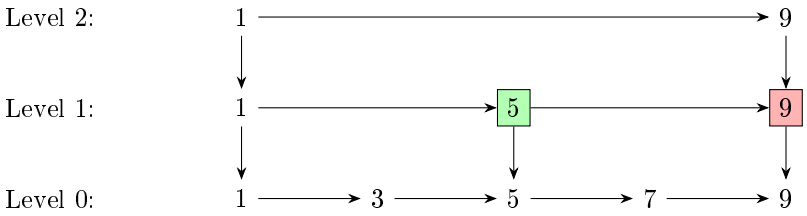


Step 2: Moved down, now check right

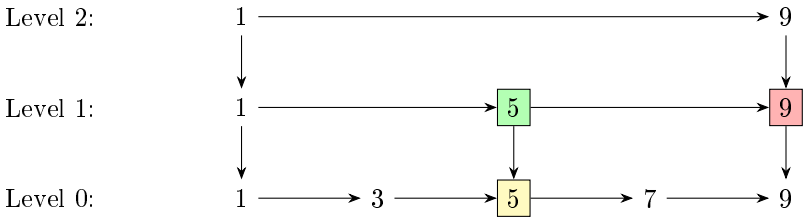


Since $5 \leq 7$, we intend to move right.

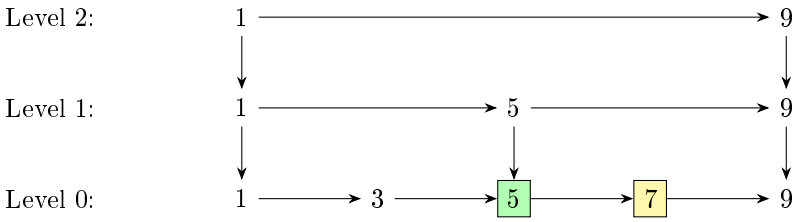
Step 3: Moved right, check right



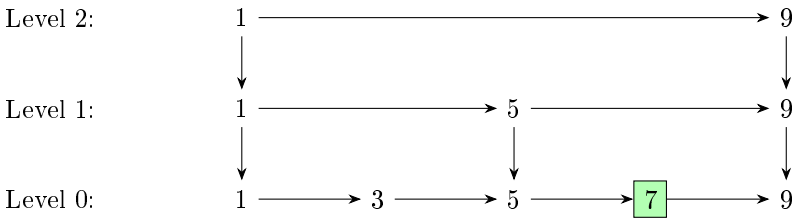
Since $9 > 7$, we intend to move down.



Step 4: Moved down, check right



Step 5: Moved right, found the target!



We move right to 7 and locate the desired element.

6.1.4 Insertion

One may wonder why skip lists use randomization at all. Could we not deterministically place

- every second element in level 1,
- every fourth element in level 2,
- every eighth element in level 3,
- and so on?

While this would indeed support logarithmic search complexity, insertions and deletions would become expensive. For example, inserting a new element near the beginning of the list changes which elements are the second, fourth, eighth, ... elements of the list.

Consequently, many levels may require rebuilding. Randomization avoids this problem by assigning heights independently to each node. Thus, updates are $\Theta(height) = \Theta(\log n)$ while the structure stays balanced in expectation.

The insertion operation in a skip list closely mirrors the search procedure. In fact, insertion is essentially a search followed by a series of pointer modifications. To insert a value x, we first perform the standard search starting from the topmost level. At each level, we move right until we either find x or encounter the first node with a key greater than x (or reach the end of the list).

If the key x is found during this traversal, then we do nothing. Otherwise, the search naturally terminates at the position where x should logically appear in the bottom level. This position is determined by the first "blocking" node whose key is greater than x, or by reaching a null pointer at the end of the list.

Once this position is identified, insertion proceeds as follows. We insert x immediately to the left of the blocking node at level 0. After placing it in the bottom layer, we decide randomly (typically with probability 1/2) whether the node should also appear in higher levels. If it is promoted, we move upward level by level, and at each level we again insert it in the same relative horizontal position, maintaining alignment with the structure discovered during the search phase.

Thus, the update process reuses the search path and only modifies pointers locally along that path. This is what ensures that skip list insertion remains logarithmic in expected time.

6.2 Treaps

Treaps are randomized balanced binary search trees. They combine two ideas:

1. The *binary search tree* property, which allows efficient searching.
2. The *heap* property, which is used to maintain balance.

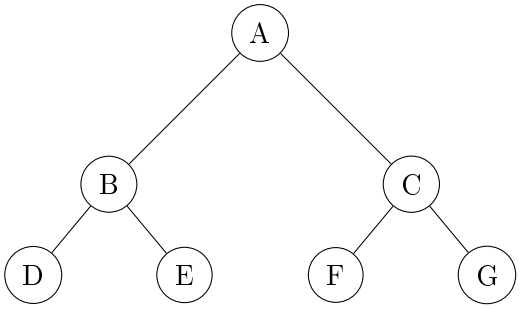
In a simple binary search tree, the insertion order matters. Certain orders can make the height $\Theta(n)$ while others can make it $\Theta(\log n)$. But if we average over all insertion orders, the height is $\Theta(\log n)$. So it seems to avoid adversarial inputs we should randomise the insertion order and the tree will be balanced in expectation! But the whole point of the data structure is we can insert elements at any time so we can not really manipulate the input order.

This brings us to the idea of assigning each element an index (called a priority) just before insertion and then whenever an element is inserted using the standard binary search tree way, we modify the existing tree to pretend as if it was inserted in increasing order of the priorities. To achieve this, we define an operation called *rotation*.

A *rotation* preserves the binary search property while "changing" the order of insertion. Thus we ensure a random insertion order and the treap remains balanced in expectation. This is a relatively simple implementation as compared to Red-Black Trees which stay balanced using rotations as well as recolorings.

6.2.1 Binary Trees

A binary tree is a rooted tree in which every node has at most two children, called the left child and the right child. For implementation purposes, the `struct Node` has three pieces of data – its own value, the pointer to its left child, and the pointer to its right child. If the child does not exist, the pointer is set to `nullptr`. We may show an empty node if there is no child for clarity in some figures.

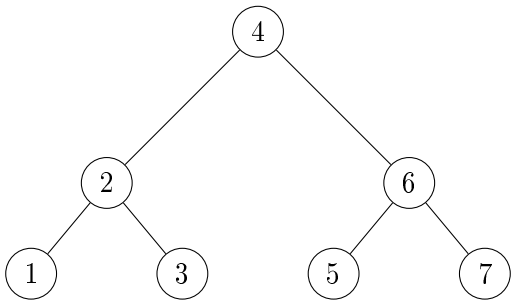


The topmost node is called the **root**. A node with no children is called a **leaf**. Every node together with all of its descendants forms a **subtree**.

6.2.2 Tree Traversal

A traversal specifies the order in which nodes are visited. The most important traversal for our purposes is the **In-Order Traversal**.

```
IOT(x)
{
    if (x is null)
        return

    IOT(x.left)
    print(x.value)
    IOT(x.right)
}

IOT(root) \\starting the recursive traversal from the root
```



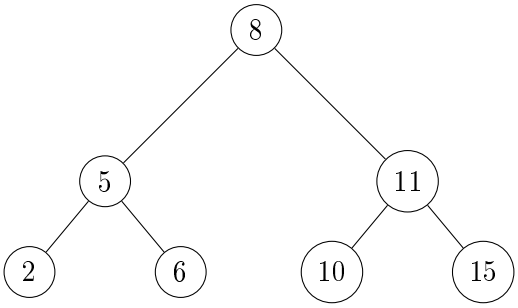
Convince yourself that the output for the above tree will be `1 2 3 4 5 6 7`. In-Order Traversal is most commonly seen in `Depth-First Search`, popularly known as `DFS`.

6.2.3 Binary Search Trees

The value stored at each node is referred to as its **key**. A binary search tree (BST) is a binary tree in which every non-leaf node satisfies the following properties:

- All keys in the left subtree are smaller than the node's key.
- All keys in the right subtree are larger than the node's key.

Any tree which has the above two properties is said to have the **BST** property.



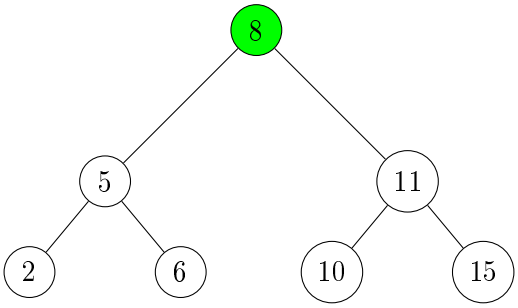
With some thought it can be proved that an In-Order Traversal of a BST lists all keys in sorted order.

Search

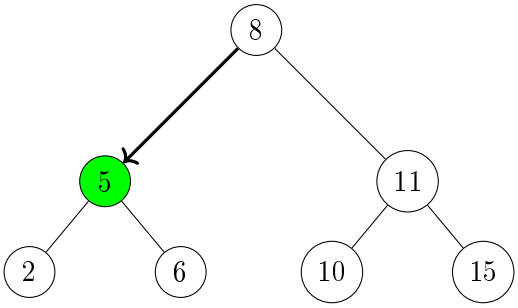
Searching for a key proceeds exactly as in binary search. At each node we compare the desired key with the current key and move either left or right. If the tree is balanced, the tree is the same as the recursion tree of a normal binary search on a sorted array ...

```
Search(key,node)
{
    if (node is null)
        return false

    if(key < node.value)
        return Search(key,node.left)
    if(node.value == key)
        return true
    if(key > node.value)
        return Search(key,node.right)
}

Search(x,root) \\Search the key x, starting from the root
```

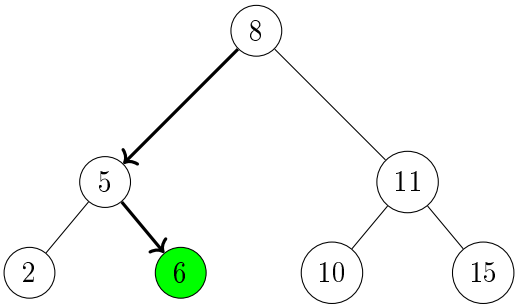
Let us say we are searching for 6 in the tree we saw earlier.



Since 8 is greater than 6, we move left.



Since 5 is less than 6, we move right.



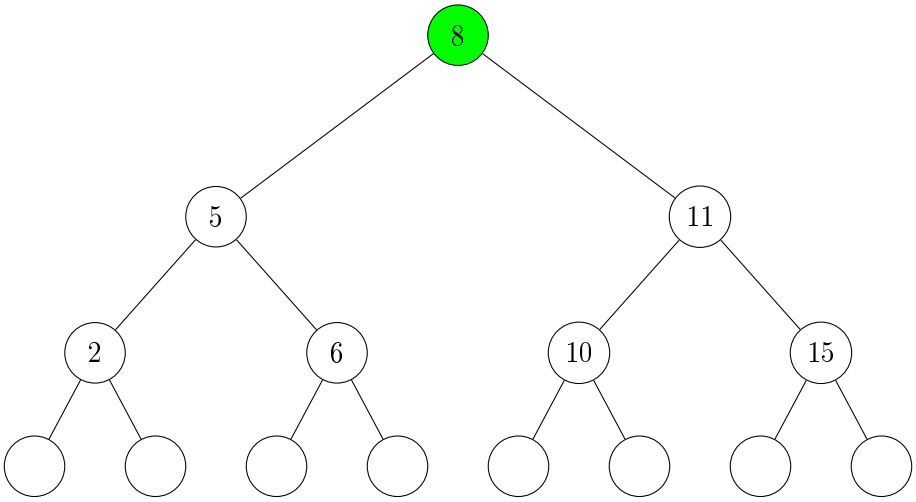
We found it! If we had instead been looking for 7, we would have taken the same path and ended up at the right child of leaf 6 (which is of course, an empty node) so we would have concluded that 7 is not in the tree.

Insertion

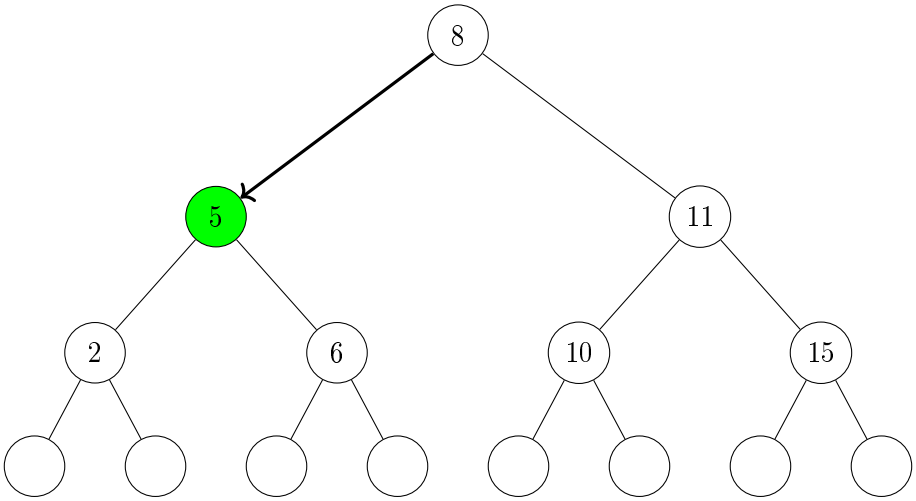
Insertion is similar to binary search. At each node we compare the key to be inserted with the current key and move either left or right. If we find the key in the tree, we do nothing. If we reach an empty node we insert the key there.

```
Insert(key,node)
{
    if (node is null)
    {
        node = new Node(value=key, left=null, right=null)
        return
    }

    if(key < node.value)
        Insert(key,node.left)

    if(node.value == key)
        return

    if(key > node.value)
        Insert(key,node.right)
}

Insert(x,root) \\Insert the key x, recurse starting from the root
```

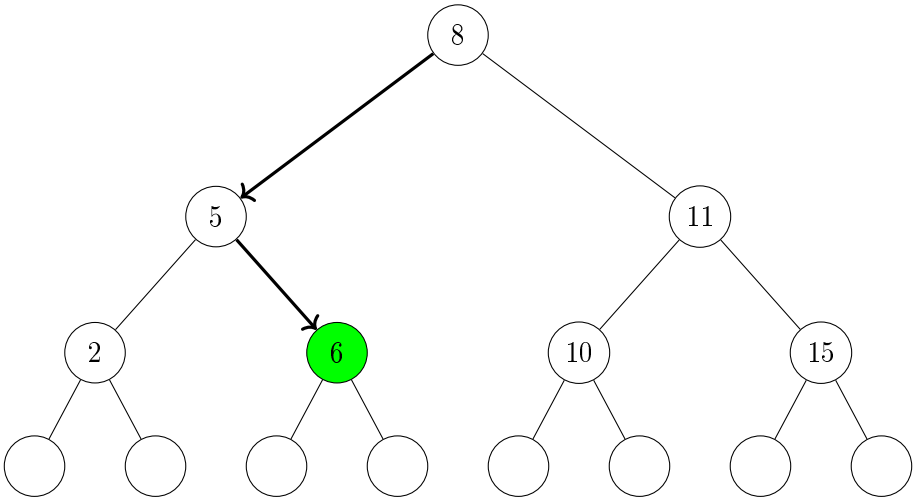
Let us say we want to insert 7 in the tree we saw earlier.



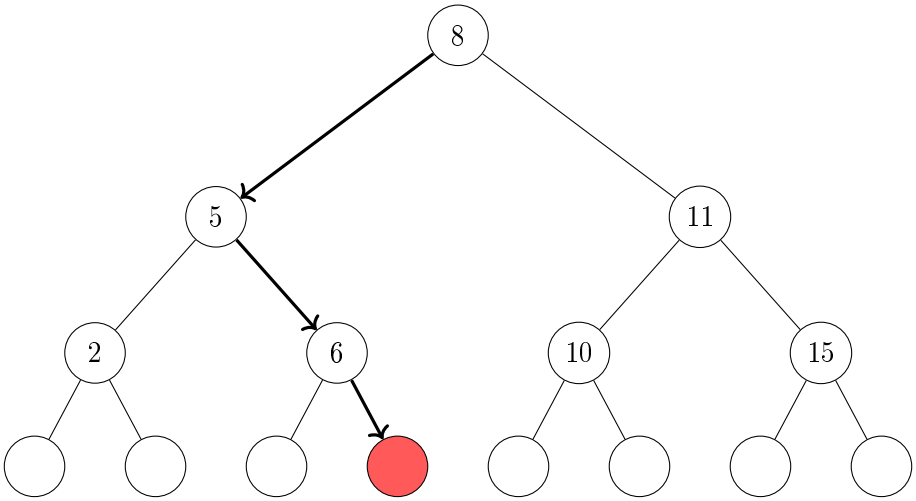
Since 8 is greater than 7, we move left.



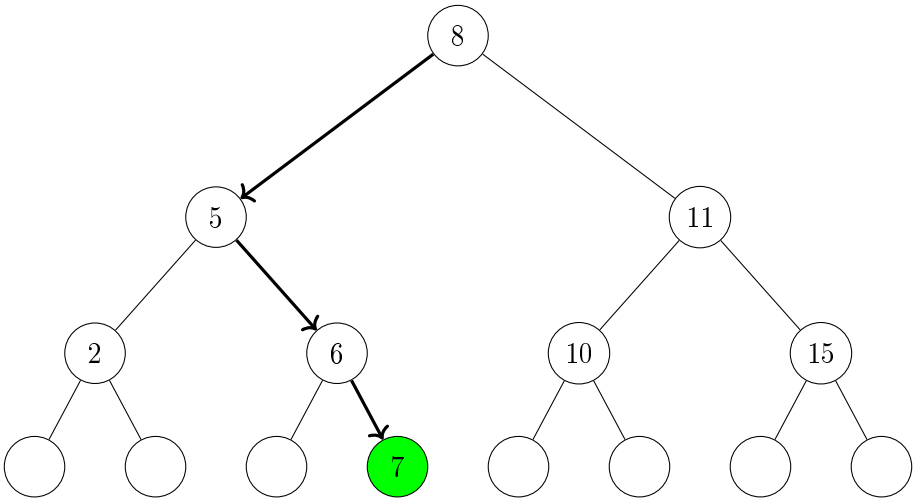
Since 5 is less than 7, we move right.



Since 6 is less than 7, we move right.

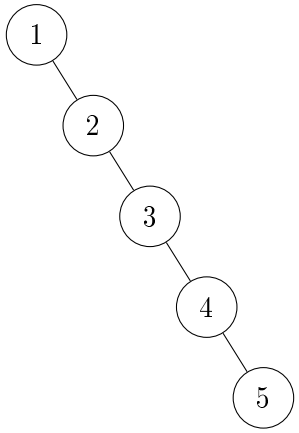


Since the node is empty, we create a new node and put 7 in it.



6.2.4 Balancing

The efficiency of a BST depends on its height. If the tree has height h, then search, insertion and deletion requires worst-case $\Theta(h)$ time. Ideally we would like $h = \Theta(\log n)$ where n is the number of stored keys. Unfortunately, a BST can become highly unbalanced. For example, inserting the keys $1, 2, 3, 4, \ldots n$ in this order produces a tree with height n.



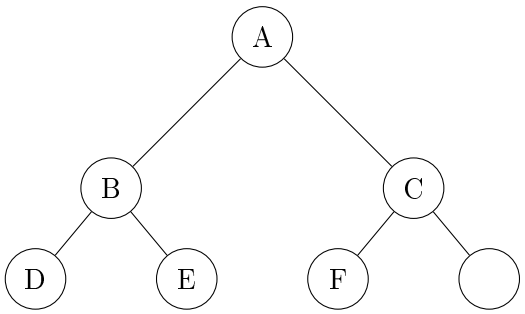
Example with n = 5

This tree is essentially a linked list. Its height is n and consequently all operations require $\Theta(n)$ time. To avoid this degeneration we need a mechanism for changing the shape of a tree while preserving the BST property.

6.2.5 Min-Heaps

A min-heap is a binary tree that satisfies two important properties:

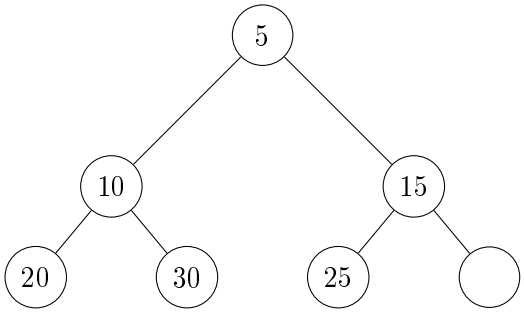
1. **Complete**: Every level is completely filled except possibly the last level, which is filled from left to right.



   The above figure shows a complete binary tree.

2. **Min-Heap Property**:

   - In a **Min-Heap**, every node is less than or equal to its children.



     The above figure shows a min-heap. It can be easily proved that the root has the smallest value in a min-heap.

A min-heap can support the `push` and `pop` operations.

Popping

The `pop()` function returns the root of the heap and removes it from the heap. To maintain the heap, we must then move some values up the tree. Logically, the smaller child should be moved up to the root. This creates a vacancy in the subtree in which the child was pushed up. That subtree now behaves like a smaller version of the original problem – a heap from which the root was removed. So we can recursively push the smaller child up until we hit empty node.

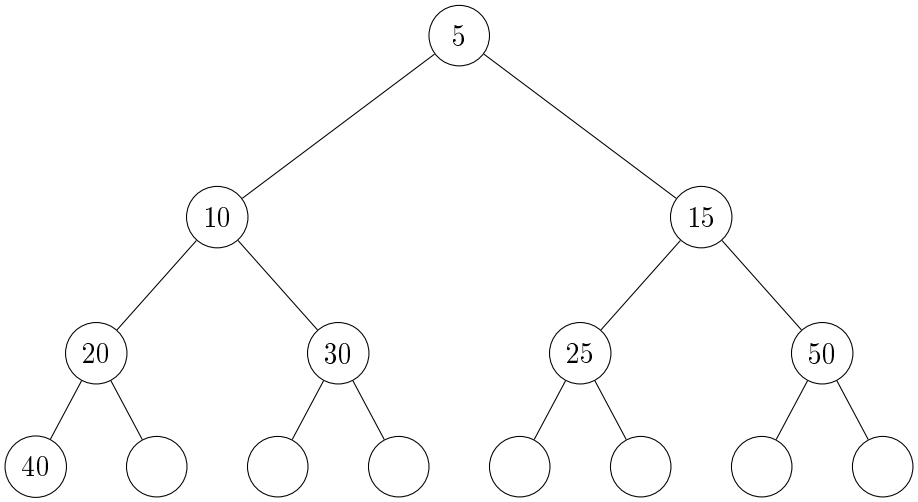
```
pop(root)
{
        ans=root.value
        root.value=null
        if(root.left < root.right)
                root.value=pop(root.leftChild)
        else
                pop(root.rightChild)
        return ans
}
```

Pushing

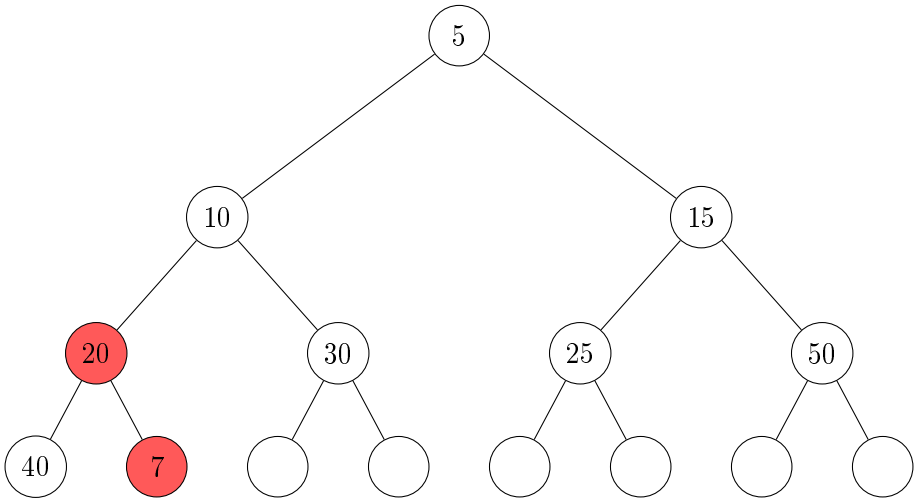
The `push()` function adds a new value to the heap by putting the new value at the bottom of the tree and then pushing it up until heap property is restored.

```
push(x)
{
        curr=bottomLeftMostNode
        curr.value=x

        while(curr.value<curr.parent.value)
        {
                swap(curr.value,curr.parent.value)
                curr=curr.parent
        }
}
```

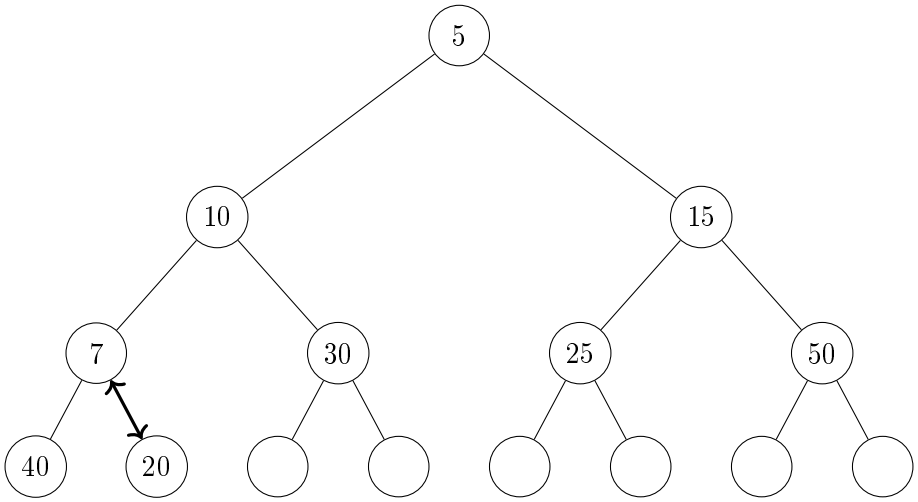
Let us insert 7 into the following min-heap.



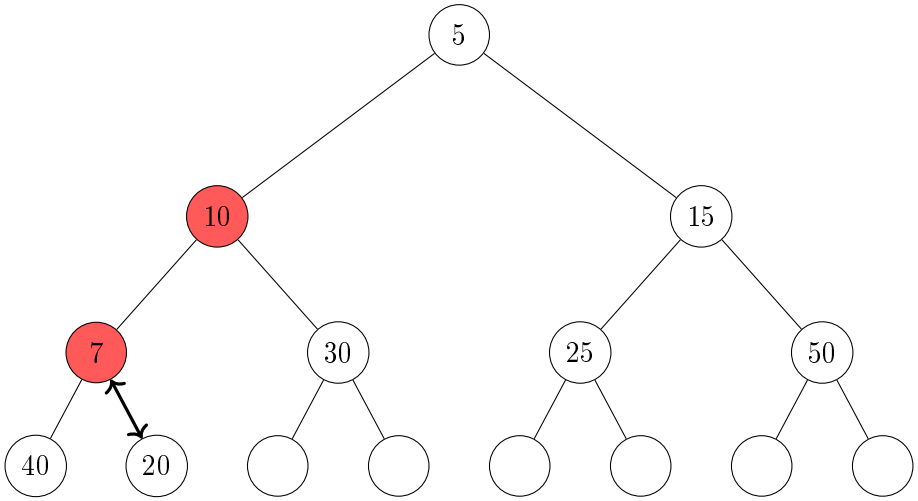
Insert 7 in bottom left most node. We detect the violation $20 > 7$.



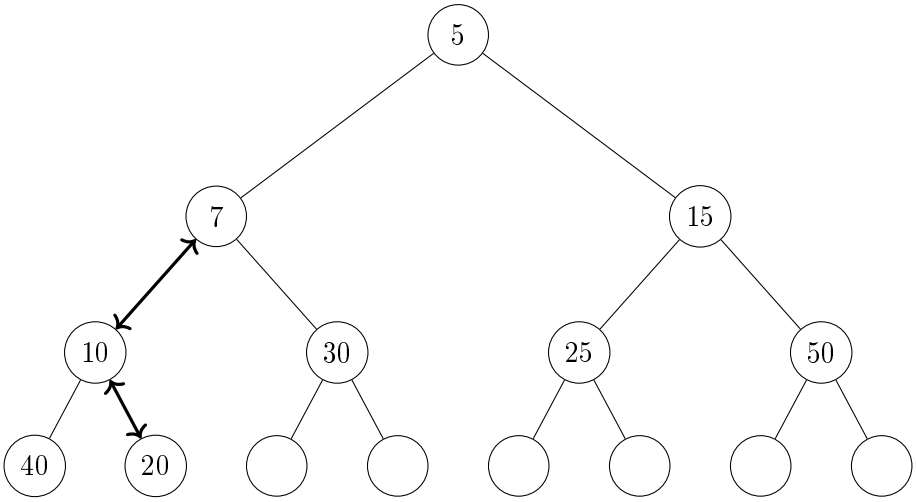
We swap them to solve the violation.



We detect the violation $10 > 7$.



We swap them to solve the violation.

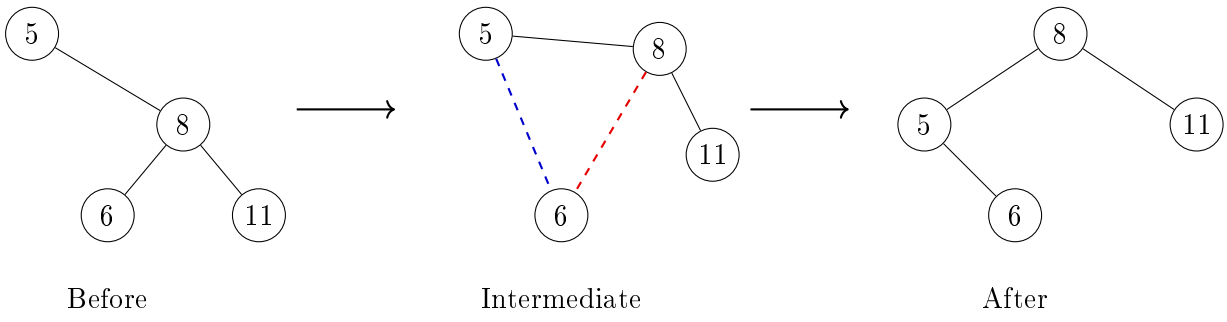


It can be proved that when there is no more violation involving the inserted value, there is no more violation in the tree. Thus the heap property is restored.

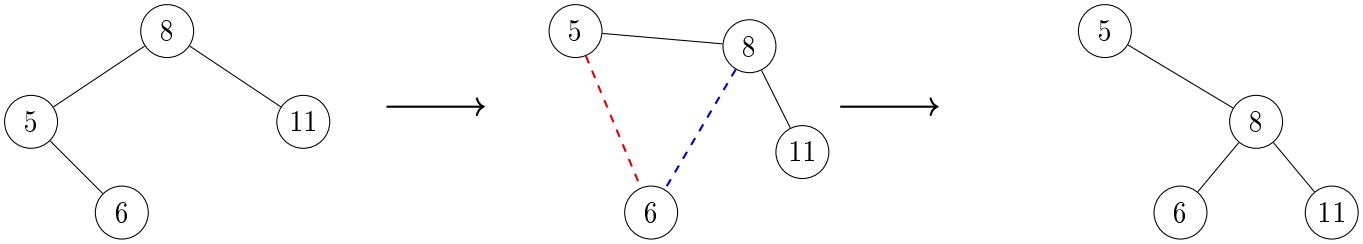
6.2.6 Rotations

These are operations that transform a tree into another tree with the same inorder traversal. From an undirected graph perspective the only change that occurs is one edge is deleted and one is added. But from a rooted tree perspective 2 father-child relationships are changed. There are two types of rotations (left and right) which are inverse of each other.

Left Rotation

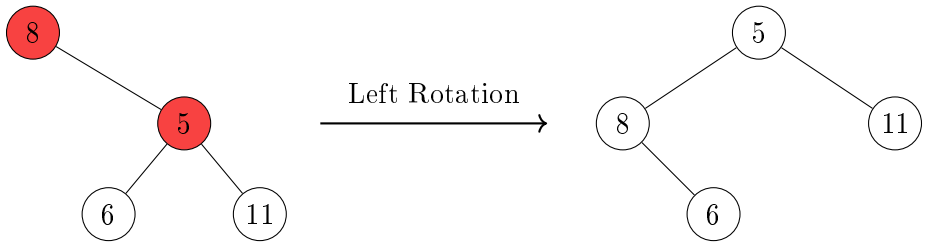


Right Rotation

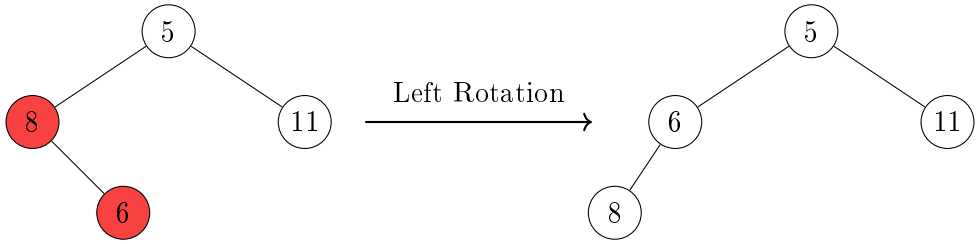


Before Intermediate After

Rotations can also fix heap violations the same way swaps did with the added benefit of preserving in-order traversal. A violation with right child and parent can be fixed with a left rotation at the parent and vice-versa.



Before After



Before After

Similar to Bubble Sort and the concept of inversions, we can look at the original tree as having 2 violations – 8 was above both 5 and 6 in the tree. Each rotation fixes exactly one of these violations just like each swap in Bubble Sort.

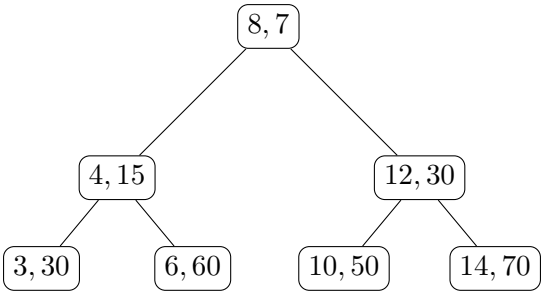
6.2.7 Treaps

Each node stores a key and along with it a random priority.

```
Node.value=(key,priority)
```

A treap simultaneously satisfies:

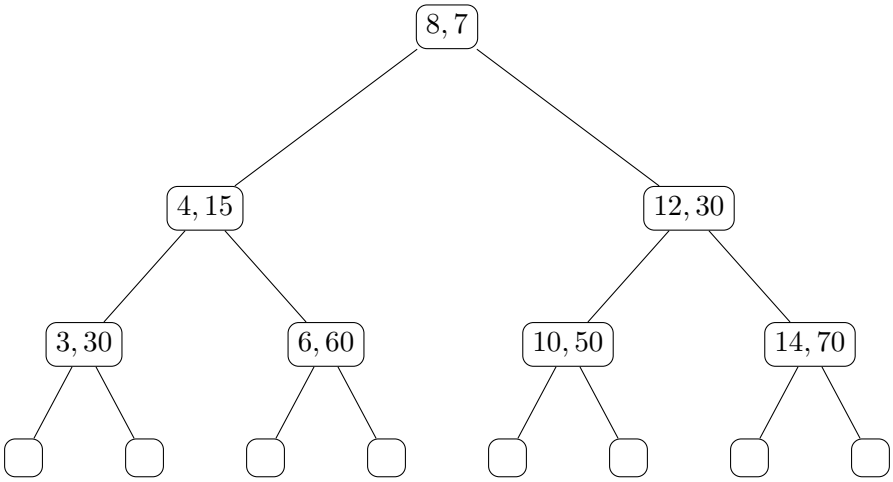
1. The **BST property** with respect to keys.
2. The **min-heap property** with respect to priorities.



The above figure shows a treap in which the first value is the key and the second is the priority.

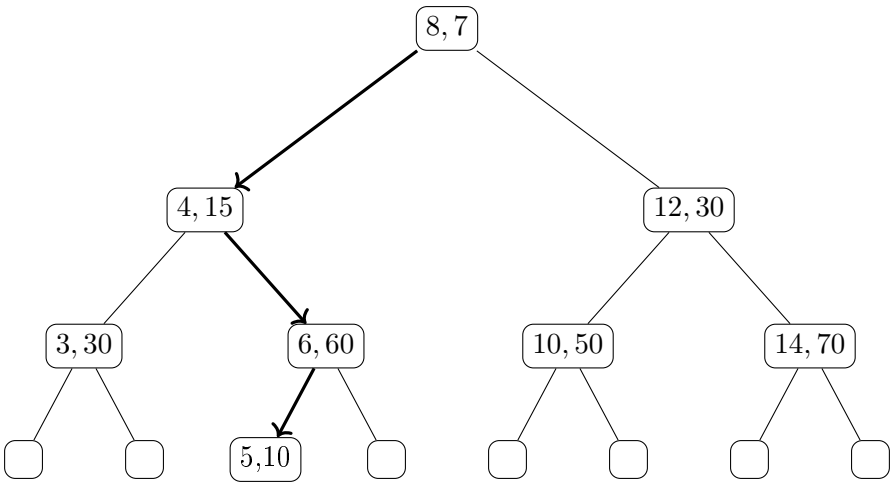
Insertion

Insertion in a treap proceeds in two phases. First, we insert the new key exactly as in a binary search tree. Then, if the heap property is violated, we restore it using rotations. Consider the following treap, where each node stores a pair (key, priority) and priorities satisfy the min-heap property.

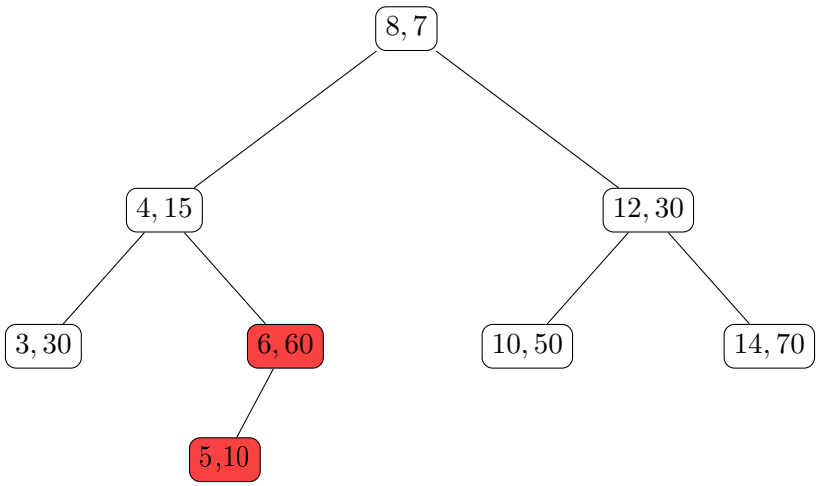


Suppose we wish to insert the node $(5, 10)$.

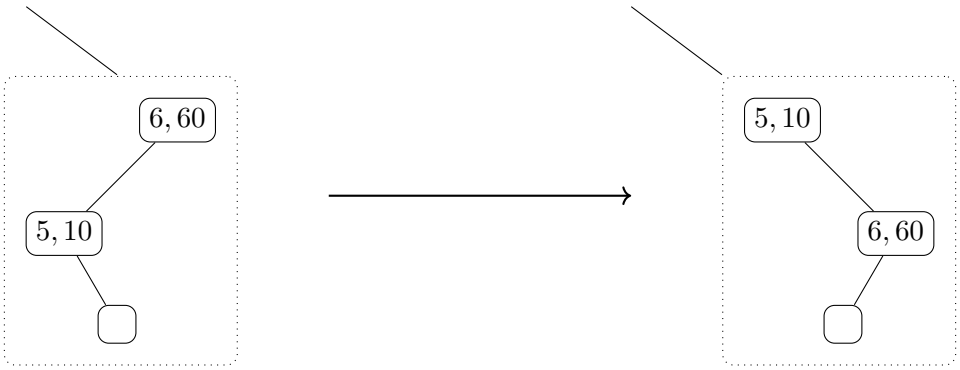
Step 1: BST Insertion Ignoring priorities, we first insert the key according to the BST property – the new node becomes the left child of $(6, 60)$.



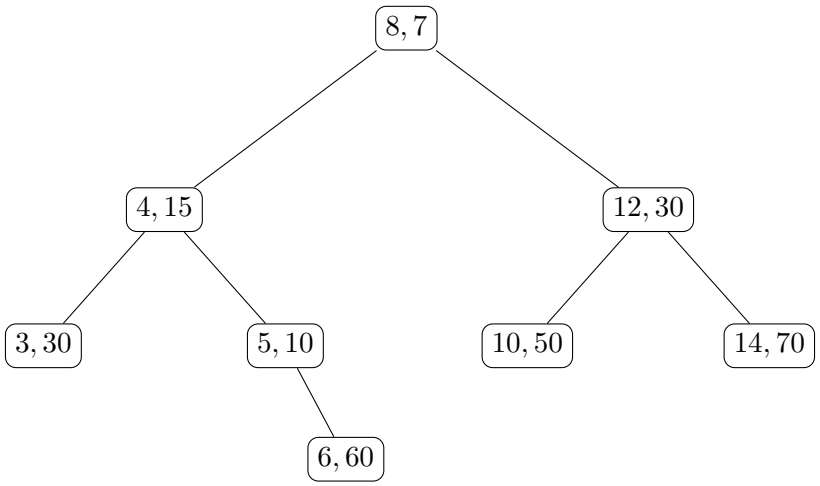
The BST property holds, but the min-heap property is violated since $10 < 60$.



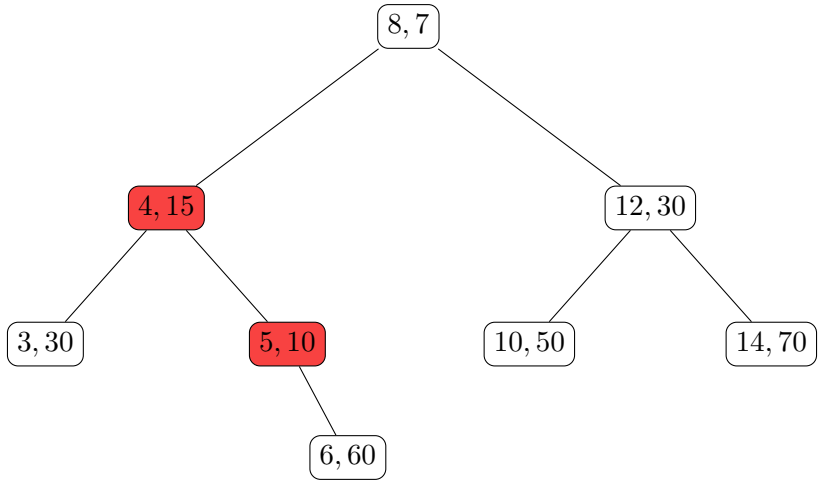
Step 2: Rotation at $(6, 60)$ Strictly speaking rotation involves three nodes, so we imagine the right child of $(5, 10)$ which is an empty node. We now perform a right rotation at $(6, 60)$.



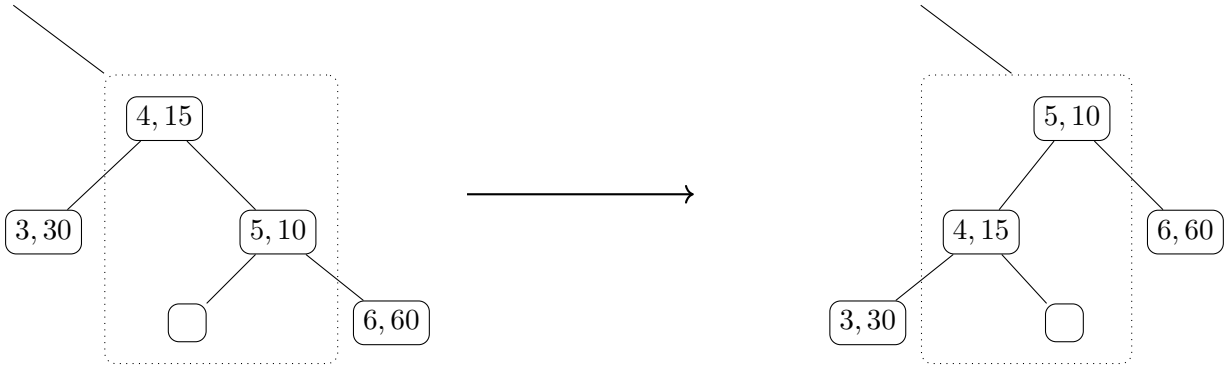
The treap now becomes



The heap property is still violated since $10 < 15$.

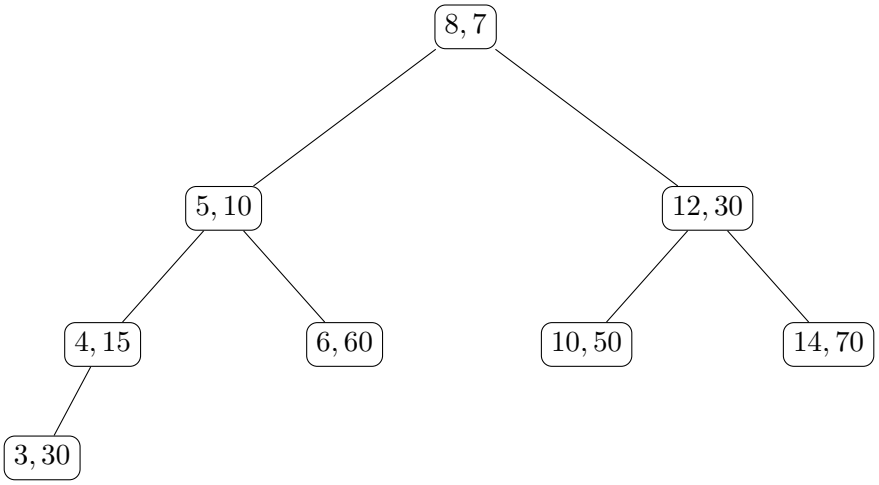


Step 3: Rotation at $(4, 15)$ We imagine the left child of $(5, 10)$ (which is an empty node) for the purposes of the rotation.



The heap property is now satisfied at every node:

$$7 < 10, \qquad 10 < 15, \qquad 10 < 60, \qquad 30 < 50, \qquad 30 < 70.$$



The heap property is now satisfied at every node. Notice that it takes at most $\Theta(H)$ time to fix a violation since the violation always involves the inserted element and in each rotation its height increases by exactly one.

Search

Searching in a treap is exactly the same as in a binary search tree – at most $\Theta(H)$ time.

Deletion

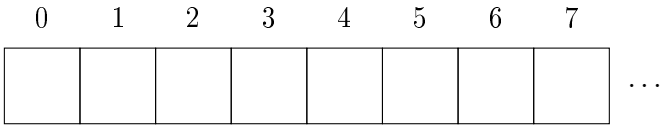
To delete an element, we must find it first. So we use the standard search to find it. Once it is found, we reverse the insert procedure by rotating *down* until it becomes a leaf. Then we can safely delete it.

6.3 Hash Tables

The need arose for a data structure to store a set of elements with efficient search, insertion and deletion operations – so we turn to hash tables!

6.3.1 The Structure

We begin with an empty array of size m. Since this is a plain old normal array of size m, we can access and modify any random element in constant time...

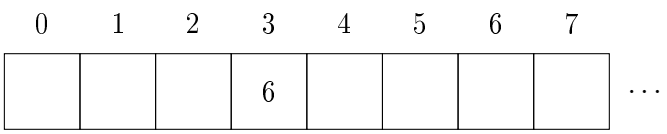


6.3.2 Insertion

We add an element x by computing $i = \mathbf{H}(x)$. For example, suppose $\mathbf{H}(6) = 3$. So we place 6 at index 3.

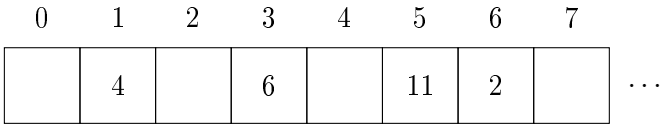
```
Insert(x)
{
    i = H(x)
    traverse Linked_List[i]
    {
        if x present
            return
    }
    add x to end of Linked_List[i]
}
```

First insertion



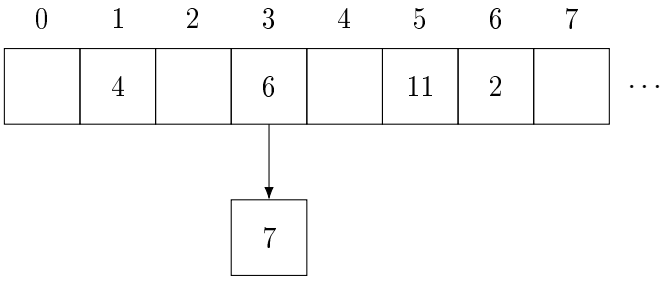
Similarly, we continue inserting elements in the same way as long as no two elements collide at the same index.

More insertions

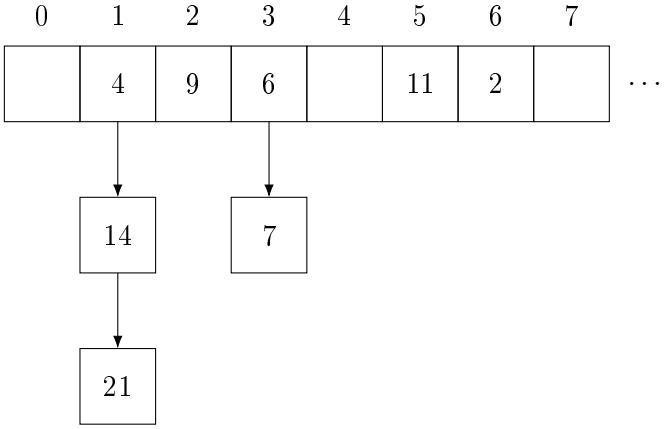


At this point, every index contains at most one element.

First *collision* Now we attempt to insert 7. Suppose $\mathbf{H}(7) = 3$. We try to place 7 at index 3, but we find 6 already present. So we extend that position into a linked list.



More collisions As more elements are inserted, collisions may occur at multiple indices, forming multiple linked lists.



Thus, each index i stores a linked list of all elements inserted till now who hash to i. Let the total number of elements inserted be n and the number of indices be m. Since on average each index has $\frac{n}{m}$ elements, the expected time of insertion is $\Theta(1 + \frac{n}{m})$.

6.3.3 Search

```
Search(x)
{
    i = H(x)
    traverse Linked_List[i]
    {
        if x found
            return true
    }
    return false
}
```

Since the expected number of elements per index is $\frac{n}{m}$, the expected time of search is $\Theta(\frac{n}{2m})$ since on average half the linked list is searched in a linear traversal.

6.3.4 Deletion

```
Delete(x)
{
    i = H(x)
    traverse Linked_List[i]
    if x found
        remove x from Linked_List[i]
}
```

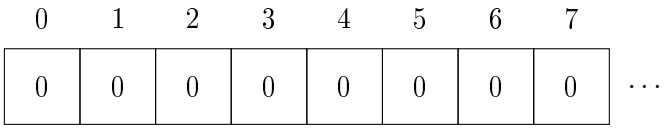
The expected time of deletion is $\Theta(\frac{n}{2m})$ since on average half the linked list is searched in a linear traversal.

6.4 Bloom Filters

Suppose we wish to store a set of elements and support the operations `insert(x)` and `contains(x)`. A straightforward solution is to store all inserted elements in an `unordered_set`. This gives expected $\Theta(1)$ insertion and lookup time, but requires memory proportional to the number of stored elements. When the number of elements becomes extremely large, memory itself may become the primary constraint. Bloom filters can achieve this tradeoff.

6.4.1 The Structure

Instead of storing the elements themselves, we store only a hash of the set. We begin with a bit array of size m initialised to all zeros.



We then choose k hash functions

$$(H_1, H_2, \ldots, H_k)$$

each mapping elements to the set of indices $\{0, 1, \ldots, m-1\}$.

6.4.2 Insertion

To insert an element x, we compute

$$(H_1(x), H_2(x), \ldots, H_k(x))$$

and set all corresponding bits to 1. For example, if $k = 3$ and the hash functions are $H_1(x) = 3,\ H_2(x) = 7,\ H_3(x) = 10$ then we set positions $3, 7$, and 10 of the bit array to 1. If while inserting we see that a bit is already 1, then we do nothing.

This operation is $\Theta(k)$ time.

6.4.3 Search

To determine whether an element x is present, we again compute

$$(H_1(x), H_2(x), \ldots, H_k(x))$$

and inspect the corresponding bits.

- If at least one of these bits is 0, then x was certainly never inserted.
- If all of these bits are 1, then we report that x is present.

The second conclusion is of course subject to false positives! It is possible that the required bits were set earlier by completely different elements.

This operation is $\Theta(k)$ time.

6.4.4 Utility of Bloom Filters

The most important property of Bloom filters is that false negatives are impossible. If the filter reports that an element is absent, then that element was definitely never inserted. The only possible error is reporting that an element is present when it is actually absent.

Interestingly, Bloom filters (as the name suggests) are used as a filter. This means that incoming queries will go through the bloom filter – if the element is absent, we can trust the result. Otherwise we can perform a more expensive lookup in the database. So it is useful to reduce the time taken to process a membership query.

It can be shown[2] that the error probability of a Bloom filter with optimal k is approximately $2^{-\frac{m \ln 2}{n}}$. For this to be small, we need m to be large and n to be small. But one might question, wasn't the whole point of the bloom filter to reduce the memory usage?

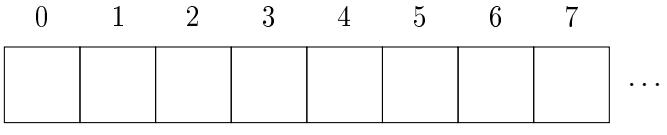
Let me clarify this with a practical example – lets say we have a database of URLs. Each URL takes 100 bytes to store. Now let us say we have 10^9 URLs – and the deterministic `set` structure has a constant factor of about 10 per node in memory. So overall that would require 10^{12} bytes of memory which is about 1 TB. In comparison, a Bloom filter of size $m = 10 \cdot n = 10^{10}$ bits would require only about 12.5 GB of memory.

6.5 Cuckoo Hashing

Recall that in ordinary hash tables, collisions are handled using linked lists. Cuckoo hashing takes a different approach. Instead of allowing multiple elements to occupy the same location, each element is given multiple possible locations and is stored in exactly one of them.

6.5.1 The Structure

We maintain an array of size m together with two hash functions (H_1, H_2).



[2] See Problems!

6.5.2 Insertion

Suppose we wish to insert an element x. Choose either of the hash functions H_1 or H_2 at random. Let this chosen hash be applied to x to obtain an index $H(x)$. If the array is empty at that index, we are done. Otherwise, let the occupant be y. We evict[3] y and place x in its position. The displaced element y must now be moved to its other possible location. If that position is occupied, another element is evicted, and the process continues.

We repeat this process until we either find an empty slot or find a cycle.

Cycles and Rehashing

In case we enter a cycle and insertion cannot be completed, choose new hash functions and rebuild the table from scratch.

Note that overall, as long as we keep n/m low, it can be shown that the expected time to insert an element is $\Theta(1)$ average time similar to hash tables.

6.5.3 Search

To determine whether an element x is present, we simply check the two possible locations $H_1(x)$ and $H_2(x)$. If x is found at either location, we report that it is present. Thus every lookup requires at most two array accesses, giving $\Theta(1)$ search time.

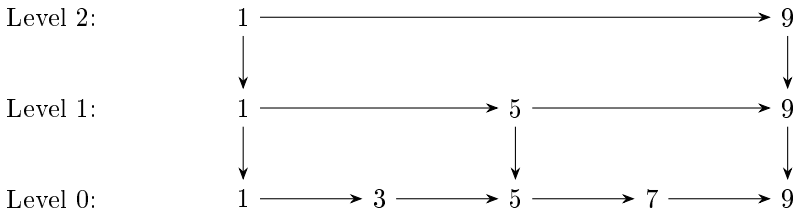
[3]This behaviour resembles the cuckoo bird which lays its eggs in another bird's nest and kicks out the occupying eggs. Hence the name ...

6.6 Problems

1 **(a)** Prove that the expected search time in a skip list is $\Theta(\log n)$. *Hint: downward moves are bounded by height, which is expected* $\Theta(\log n)$*.*

(b) Deduce similarly that insertion takes expected $\Theta(\log n)$ time.

(c) Try inserting 8 into the following skip list using the procedure we described earlier:



(d) Can you think of a $\Theta(\log n)$ deletion strategy for skip lists?

2 Consider n,m,k to be number of inserted elements, the size of bit array and the number of hash functions respectively in a Bloom filter. Assume the hash functions are independent.

(a) Show that the probability of a particular bit in the array being set is $\left(1-\left(1-\frac{1}{m}\right)^{kn}\right)$ and thus the probability of a false positive is $\left(1-\left(1-\frac{1}{m}\right)^{kn}\right)^{k}$

(b) Using suitable large m, n approximations, show that the optimal number of hash functions is $k=\frac{m}{n}\ln 2$

(c) Put the above results together to show that the probability of a false positive is approximately $2^{-\frac{m\ln 2}{n}}$.

3 Consider a Binary-Search Tree in which n distinct elements are inserted in a random order. Without loss of generality, consider the elements to be $1, 2, \ldots, n$. Let the random variable H_i be the height when i elements are inserted into the BST. Of course, $H_0 = 0$ and $H_1 = 1$.

(a) Prove that $H_n \geq \log_2(n+1) - 1$

(b) Use the definition of BST on the root to show that

$$(H_n|\ root = i) = 1 + max(H_{i-1}, H_{n-i})$$

(c) Use the substitution $Y_i = 2^{H_i}$ and take expectation both sides to show that

$$\mathbb{E}[Y_n|\ root = i] = 2 \cdot \mathbb{E}[max(Y_{i-1}, Y_{n-i})]$$

(d) Use the inequality $max(a, b) \leq a + b$ and the law of total expectation to show that

$$\mathbb{E}[Y_n] \leq \frac{4}{n} \cdot \sum_{i=1}^{n} \mathbb{E}[Y_{i-1}]$$

(e) Substitute S_n as the sum of $\mathbb{E}[Y_i]$ from 0 to n and show that

$$S_n \leq S_{n-1}\left(1 + \frac{4}{n}\right)$$

(f) Use Telescoping Product and the value of $S_0 = \mathbb{E}[Y_0] = \mathbb{E}[2^{H_0}] = 1$ to show that

$$S_n \leq \binom{n}{4}$$

(g) Use the expectation form of Jensen's inequality and the result from **(f)** to show that

$$\mathbb{E}[H_n] \leq \log_2 \left(\binom{n-1}{3} \right)$$

(h) Use the result from **(a)** and **(g)** to show that

$$\mathbb{E}[H_n] = \Theta(\log n)$$